rqbit 통계 수집 및 모니터링 시스템 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 통계 수집 및 모니터링 시스템을 분석합니다. rqbit은 다층적인 통계 수집 아키텍처와 실시간 모니터링 기능을 통해 성능 분석과 시스템 관찰 가능성을 제공합니다.

1. 성능 메트릭 수집 및 Prometheus 통합

1.1 통계 생성 매크로 시스템

rqbit은 매크로를 사용하여 일관된 통계 수집 구조를 생성합니다:

```
macro_rules! gen_stats {
($atomic_name:ident $snapshot_name:ident, [$($stat_name:ident $stat_ty:tt),*], [
#[derive(Debug, Default)]
pub  struct $atomic_name {
$(
pub $stat_name: stype!(atomic $stat_ty),
)*
$(
pub $nested_field_name: $nested_atomic_name,
)*
}
```

```
impl $atomic_name {
pub  fn snapshot(&self) -> $snapshot_name {
$snapshot_name {
$(
$stat_name: self.$stat_name.load(std::sync::atomic::Ordering::Relaxed),
)*
$(
$nested_field_name: self.$nested_field_name.snapshot(),
)*
}
}
}
};
}
```

매크로 시스템 특징:

- **원자적 카운터**: 멀티스레드 환경에서 안전한 통계 수집
- **스냅샷 생성**: 일관된 시점의 통계 데이터 캡처
- **중첩 구조**: 계층적 통계 구조 지원
- **타입 안전성**: 컴파일 타임 타입 검증

1.2 세션 레벨 통계

```rust
// 세션 통계 정의
gen_stats!(SessionCountersAtomic SessionCountersSnapshot, [
fetched_bytes u64,
uploaded_bytes u64,
blocked_incoming u64,
blocked_outgoing u64
], []);


pub struct SessionStats {
pub counters: SessionCountersAtomic,
pub peers: Arc<AggregatePeerStatsAtomic>,
pub down_speed_estimator: SpeedEstimator,
pub up_speed_estimator: SpeedEstimator,
pub startup_time: Instant,
}
```

세션 통계 특징:

- **바이트 카운터**: 다운로드/업로드 바이트 수 추적
- **연결 차단**: 차단된 인바운드/아웃바운드 연결 수
- **속도 추정**: 실시간 다운로드/업로드 속도 계산
- **업타임**: 세션 시작 시간부터의 경과 시간

1.3 피어 통계 시스템

```
// 피어 통계 정의
gen_stats!(AggregatePeerStatsAtomic AggregatePeerStats, [
queued u32,
connecting u32,
live u32,
live_tcp u32,
live_utp u32,
live_socks u32,
seen u32,
dead u32,
not_needed u32,
steals u32
], []);


impl AggregatePeerStatsAtomic {
fn counter(&self, state: &PeerState) -> &AtomicU32 {
match state {
PeerState::Connecting(_) => &self.connecting,
PeerState::Live(_) => &self.live,
PeerState::Queued => &self.queued,
PeerState::Dead => &self.dead,
PeerState::NotNeeded => &self.not_needed,
```

```
}
}

pub(crate) fn inc(&self, state: &PeerState) {
if let PeerState::Live(l) = state {
atomic_inc(self.live_kind_counter(l));
}
atomic_inc(self.counter(state));
}
}
```

피어 통계 특징:

- **상태별 분류**: 피어 상태에 따른 자동 분류
- **연결 타입별**: TCP, uTP, SOCKS 연결 타입별 통계
- **동적 업데이트**: 피어 상태 변경 시 자동 통계 업데이트
- **청크 스틸링**: 피어 간 청크 스틸링 횟수 추적

1.4 Prometheus 메트릭 출력

```
impl SessionStatsSnapshot {
pub fn as_prometheus(&self, mut out: &mut String) {
use core::fmt::Write;

macro_rules! m {
```

```
($type:ident, $name:ident, $value:expr) => {{
writeln!(&mut out, concat!("# TYPE ", stringify!($name), " ", stringify!($type))
writeln!(&mut out, concat!(stringify!($name), " {}"), $value).unwrap();
}};
}


m!(counter, rqbit_fetched_bytes, self.counters.fetched_bytes);
m!(counter, rqbit_uploaded_bytes, self.counters.uploaded_bytes);
m!(counter, rqbit_blocked_incoming, self.counters.blocked_incoming);
m!(counter, rqbit_blocked_outgoing, self.counters.blocked_outgoing);
m!(gauge, rqbit_download_speed_bytes, self.download_speed.as_bytes());
m!(gauge, rqbit_upload_speed_bytes, self.upload_speed.as_bytes());
m!(gauge, rqbit_uptime_seconds, self.uptime_seconds);
// 피어 상태별 메트릭
writeln!(&mut out, "# TYPE rqbit_peers_live gauge").unwrap();
writeln!(&mut out, "rqbit_peers_live{{kind=\"tcp\"}} {}", self.peers.live_tcp).u
writeln!(&mut out, "rqbit_peers_live{{kind=\"utp\"}} {}", self.peers.live_utp).u
writeln!(&mut out, "rqbit_peers_live{{kind=\"socks\"}} {}", self.peers.live_sock
}
}
```

Prometheus 통합 특징:

- **표준 형식**: Prometheus 표준 메트릭 형식 준수

- **메트릭 타입**: Counter와 Gauge 타입 지원
- **레이블 지원**: 연결 타입별 레이블 분류
- **자동 생성**: 매크로를 통한 메트릭 자동 생성

2. 실시간 통계 계산 및 히스토그램 관리

2.1 속도 추정 시스템

```
impl Session {
pub(crate) fn start_speed_estimator_updater(self: &Arc<Self>) {
self.spawn(
debug_span!(parent: self.rs(), "speed_estimator"),
"speed_estimator",
{
let s = Arc::downgrade(self);
async move {
let mut i = tokio::time::interval(Duration::from_secs(1));
loop {
i.tick().await;
let s = s.upgrade().context("session is dead")?;
let now = Instant::now();
let fetched = s.stats.counters.fetched_bytes.load(Ordering::Relaxed);
let uploaded = s.stats.counters.uploaded_bytes.load(Ordering::Relaxed);
// 속도 추정기 업데이트
```

```
s.stats.down_speed_estimator.add_snapshot(fetched, None, now);
s.stats.up_speed_estimator.add_snapshot(uploaded, None, now);
}
}
},
)
}
}
```

속도 추정 특징:

- **주기적 업데이트**: 1초 간격으로 속도 계산
- **윈도우 기반**: 5초 윈도우를 사용한 이동 평균
- **약한 참조**: 메모리 누수 방지를 위한 약한 참조 사용
- **실시간 계산**: 실시간 다운로드/업로드 속도 제공

2.2 토렌트 레벨 통계

```rust
#[derive(Serialize, Debug)]
pub struct TorrentStats {
pub state: TorrentStatsState,
pub file_progress: Vec<u64>,
pub error: Option<String>,
pub progress_bytes: u64,
pub uploaded_bytes: u64,
```

```rust
pub total_bytes: u64,
pub finished: bool,
pub live: Option<LiveStats>,
}

#[derive(Serialize, Default, Debug)]
pub struct LiveStats {
pub snapshot: StatsSnapshot,
pub average_piece_download_time: Option<Duration>,
pub download_speed: Speed,
pub upload_speed: Speed,
pub time_remaining: Option<DurationWithHumanReadable>,
}
```

토렌트 통계 특징:

- **진행률 추적**: 바이트 단위 진행률 및 파일별 진행률
- **상태 관리**: 초기화, 라이브, 일시정지, 오류 상태 추적
- **ETA 계산**: 남은 시간 추정 및 인간 친화적 표시
- **조각 통계**: 평균 조각 다운로드 시간 계산

2.3 인간 친화적 표시 시스템

```rust
impl TorrentStats {
pub fn progress_percent_human_readable(&self) -> impl std::fmt::Display {
```

```
struct Percents { progress: u64, total: u64 }

impl std::fmt::Display for Percents {

fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {

if self.total == 0 {

return write!(f, "N/A");

}

let pct = self.progress as f64 / self.total as f64 * 100f64;

write!(f, "{pct:.2}%")

}

}

Percents { progress: self.progress_bytes, total: self.total_bytes }

}

}



fn format_seconds_to_time(seconds: u64, f: &mut core::fmt::Formatter<'_>) -> cor

let hours = seconds / 3600;

let minutes = (seconds % 3600) / 60;

let seconds = seconds % 60;



if hours > 0 {

write!(f, "{}h {}m", hours, minutes)

} else if minutes > 0 {

write!(f, "{}m {}s", minutes, seconds)
```

```
} else {
write!(f, "{}s", seconds)
}
}
```

표시 시스템 특징:

- **백분율 표시**: 소수점 둘째 자리까지 정확한 진행률
- **크기 표시**: 바이너리 단위(MiB, GiB)를 사용한 크기 표시
- **시간 표시**: 시/분/초 단위의 직관적 시간 표시
- **JSON 직렬화**: API 응답을 위한 구조화된 데이터

3. 성능 병목 지점 식별 및 알림 시스템

3.1 HTTP API 모니터링 엔드포인트

```
pub fn make_api_router(state: ApiState) -> Router {
let mut api_router = Router::new()
.route("/stats", get(torrents::h_session_stats))
.route("/torrents/{id}/stats/v1", get(torrents::h_torrent_stats_v1))
.route("/torrents/{id}/peer_stats", get(torrents::h_peer_stats))
.route("/torrents/{id}/peer_stats/prometheus", get(torrents::h_peer_stats_promet
.route("/dht/stats", get(dht::h_dht_stats))
.route("/dht/table", get(dht::h_dht_table));
```

```
// Prometheus 메트릭 엔드포인트
#[cfg(feature = "prometheus")]
if let Some(handle) = prometheus_handle.take() {
let session = state.api.session().clone();
main_router = main_router.route(
"/metrics",
get(move || async move {
let mut metrics = handle.render();
session.stats_snapshot().as_prometheus(&mut metrics);
metrics
}),
);
}
}
```

모니터링 엔드포인트 특징:

- **세션 통계**: 전역 세션 통계 조회
- **토렌트 통계**: 개별 토렌트 상세 통계
- **피어 통계**: 피어별 성능 통계
- **DHT 통계**: DHT 노드 및 라우팅 테이블 상태
- **Prometheus**: 표준 메트릭 형식 지원

3.2 성능 병목 감지

```
// 피어 상태 추적을 통한 병목 감지
impl PeerStates {
pub fn is_peer_not_interested_and_has_full_torrent(
&self,
handle: PeerHandle,
total_pieces: usize,
) -> bool {
self.with_live(handle, |live| {
!live.peer_interested && live.has_full_torrent(total_pieces)
}).unwrap_or(false)
}


pub(crate) fn on_steal(&self, from_peer: SocketAddr, to_peer: SocketAddr, stolen
// 스틸링 통계 업데이트
self.with_peer(to_peer, |p| {
atomic_inc(&p.stats.counters.times_i_stole);
});
self.with_peer(from_peer, |p| {
atomic_inc(&p.stats.counters.times_stolen_from_me);
});
self.stats.inc_steals();
self.session_stats.inc_steals();
```

```
}

}
```

병목 감지 특징:

- **피어 관심도**: 피어의 관심 상태 추적
- **청크 스틸링**: 비효율적 피어로부터의 청크 재할당
- **연결 상태**: 연결 실패 및 타임아웃 추적
- **처리량 모니터링**: 피어별 처리량 비교

3.3 동적 성능 조정

```
// 속도 기반 ETA 계산

impl From<&TorrentStateLive> for LiveStats {

fn from(live: &TorrentStateLive) -> Self {

let snapshot = live.stats_snapshot();

let down_estimator = live.down_speed_estimator();

let up_estimator = live.up_speed_estimator();


Self {

average_piece_download_time: snapshot.average_piece_download_time(),

snapshot,

download_speed: down_estimator.mbps().into(),

upload_speed: up_estimator.mbps().into(),

time_remaining: down_estimator.time_remaining().map(DurationWithHumanReadable),
```

```
}

}

}
```

4. 통합 모니터링 아키텍처

4.1 계층적 통계 구조

```
세션 레벨 통계

├── 전역 바이트 카운터

├── 전역 속도 추정

├── 피어 집계 통계

└── 시스템 업타임


토렌트 레벨 통계

├── 진행률 및 상태

├── 파일별 진행률

├── 토렌트별 속도

└── ETA 계산


피어 레벨 통계

├── 연결 상태별 분류

├── 프로토콜별 분류
```

```
├── 개별 피어 성능
└── 청크 스틸링 통계
```

4.2 실시간 업데이트 메커니즘

- **원자적 연산**: 락 없는 통계 업데이트
- **주기적 스냅샷**: 일관된 시점의 데이터 제공
- **이벤트 기반**: 상태 변경 시 즉시 통계 업데이트
- **백그라운드 처리**: 통계 계산이 주 작업에 영향 없음

4.3 외부 시스템 통합

- **Prometheus**: 표준 메트릭 형식으로 외부 모니터링 시스템 연동
- **HTTP API**: RESTful API를 통한 프로그래매틱 접근
- **JSON 직렬화**: 웹 UI 및 외부 도구와의 호환성
- **로그 스트리밍**: 실시간 로그 모니터링 지원

5. 성능 및 확장성 특성

5.1 메모리 효율성

- **원자적 카운터**: 최소한의 메모리 오버헤드
- **스냅샷 기반**: 필요 시에만 데이터 복사
- **약한 참조**: 순환 참조 방지로 메모리 누수 없음
- **매크로 생성**: 컴파일 타임 최적화

5.2 성능 영향 최소화

- **비동기 처리**: 통계 수집이 주 작업을 블로킹하지 않음
- **배치 업데이트**: 여러 통계를 한 번에 업데이트
- **지연 계산**: 요청 시에만 복잡한 계산 수행
- **캐시 친화적**: 지역성을 고려한 데이터 구조

5.3 확장성

- **수평 확장**: 토렌트 수 증가에 선형적 확장
- **수직 확장**: 시스템 리소스에 비례한 성능
- **모듈화**: 독립적인 통계 모듈로 유지보수성 향상
- **표준 호환**: 업계 표준 모니터링 도구와 호환

이러한 포괄적인 통계 수집 및 모니터링 시스템은 rqbit의 성능 분석, 문제 진단, 그리고 시스템 최적화를 위한 핵심 인프라를 제공합니다.